Simplifying mocks with interface embedding

Tiago Melo
Senior software engineer

tiagoharris@gmail.com

Use case: cache

Simple Post service that uses Redis to increment the number of posts of a given user

1. Install the modules

```
$ github.com/redis/go-redis/v9
$ github.com/pkg/errors
$ github.com/stretchr/testify
```

2. cache/cache.go: Cache service interface

```go
package cache

import (
    "context"
    "time"
)

// CacheService defines the interface for a cache.
type CacheService interface {
    Get(ctx context.Context, key string) (string,
error)
    Set(ctx context.Context, key string, value int,
expiration time.Duration) error
    Incr(ctx context.Context, key string) (int64,
error)
}
```

3. cache/redis.go: Redis cache service implementation

```go
package cache

import (
	"context"
	"fmt"
	"time"

	"github.com/pkg/errors"
	"github.com/redis/go-redis/v9"
)

// redisCache provides a Redis implementation of the CacheService
interface.
type redisCache struct {
	redisClient *redis.Client
}

// Compile-time assertion to ensure redisCache implements CacheService.
// This check aids in easier mocking of CacheService during unit tests.
var _ CacheService = (*redisCache)(nil)
```

3. cache/redis.go: Redis cache service implementation

```
var _ InterfaceName = (*ConcreteType)(nil)
```

This line effectively performs a compile-time check, ensuring that **ConcreteType** correctly implements all functions declared in **InterfaceName**.

An interface might declare multiple functions, not all of which are relevant to every test. Writing a complete mock that implements every function of an interface can be tedious and unnecessary. Here's where the technique of embedding interfaces into mock types becomes invaluable.

3. cache/redis.go: Redis cache service implementation

```go
func (rc *redisCache) Incr(ctx context.Context, key string) (int64,
error) {
	intCmd := rc.redisClient.Incr(ctx, key)
	if err := intCmd.Err(); err != nil {
		return 0, errors.Wrapf(err, `incr key "%s"`, key)
	}
	return intCmd.Val(), nil
}

func (rc *redisCache) Get(ctx context.Context, key string) (string,
error) {
	getCmd := rc.redisClient.Get(ctx, key)
	if err := getCmd.Err(); err != nil {
		if err == redis.Nil {
			return "", nil
		}
		return "", errors.Wrapf(err, `get key "%s"`, key)
	}
	return getCmd.Val(), nil
}
```

3. cache/redis.go: Redis cache service implementation

```
func (rc *redisCache) Set(ctx context.Context, key string, value int,
expiration time.Duration) error {
	if err := rc.redisClient.Set(ctx, key, value, expiration).Err();
err != nil {
		return errors.Wrapf(err, `set key "%v" with value "%v" and
expiration %v`, key, value, expiration)
	}
	return nil
}

func NewRedisCache(ctx context.Context, host, port string)
(CacheService, error) {
	client := redis.NewClient(&redis.Options{
		Addr: fmt.Sprintf("%s:%s", host, port),
	})
	statusCmd := client.Ping(ctx)
	if err := statusCmd.Err(); err != nil {
		return nil, errors.Wrap(err, "pinging redis instance")
	}
	return &redisCache{client}, nil
}
```

4. post/post.go: Post service

```go
// Post represents a blog post or user submission.
type Post struct {
    Title   string
    Content string
}

// PostService handles operations related to posts.
type PostService struct {
    cache cache.CacheService
}

// NewPostService creates a new instance of PostService.
func NewPostService(cache cache.CacheService) *PostService {
    return &PostService{cache: cache}
}
```

4. post/post.go: Post service

```
// CreatePost simulates creating a new post and increments
the total post count.
func (p *PostService) CreatePost(ctx context.Context, post
*Post) error {
    const key = "postCount"
    // increment the post count in the cache.
    _, err := p.cache.Incr(ctx, key)
    if err != nil {
        return errors.Wrapf(err, `incrementing key "%s"`,
key)
    }
    // logic to persist post.
    return nil
}
```

5. post/post_test.go: testing CreatePost

```go
type mockCacheService struct {
    cache.CacheService

    incrErr error
}

func (m *mockCacheService) Incr(ctx context.Context, key
string) (int64, error) {
    return 1, m.incrErr
}
```

The **mockCacheService** struct embeds the **CacheService** interface from the **cache** package. Embedding the interface allows **mockCacheService** to inherit all functions of the **CacheService** interface. This setup means that **mockCacheService** automatically fulfills the **CacheService** interface, and it can be used wherever CacheService is expected.

5. post/post_test.go: testing CreatePost

```
type mockCacheService struct {
    cache.CacheService

    incrErr error
}

func (m *mockCacheService) Incr(ctx context.Context, key
string) (int64, error) {
    return 1, m.incrErr
}
```

By embedding, **mockCacheService** inherits all methods of **CacheService**, and you only need to override the functions you want to control or simulate specific behaviors for in your tests - in our case, just **Incr** function.

5. post/post_test.go: testing CreatePost

```
func TestCreatePost(t *testing.T) {
	testCases := []struct {
		name          string
		mockClosure   func(m *mockCacheService)
		expectedError error
	}{
		{
			name:        "happy path",
			mockClosure: func(m *mockCacheService) {},
		},
		{
			name: "error when incrementing postCount key in cache",
			mockClosure: func(m *mockCacheService) {
				m.incrErr = errors.New("incr error")
			},
			expectedError: errors.New(`incrementing key "postCount":
incr error`),
		},
	}

	// see the test logic in the next slide
}
```

5. post/post_test.go: testing CreatePost

```
    for _, tc := range testCases {
        t.Run(tc.name, func(t *testing.T) {
            m := new(mockCacheService)
            tc.mockClosure(m)
            ps := NewPostService(m)
            err := ps.CreatePost(context.TODO(), &Post{})
            if err != nil {
                if tc.expectedError == nil {
                    t.Fatalf("expected no error, got %v", err)
                }
                require.Equal(t, tc.expectedError.Error(),
err.Error())
            } else {
                if tc.expectedError != nil {
                    t.Fatalf("expected error to be %v, got nil",
tc.expectedError)
                }
            }
        })
    }
```

5. post/post_test.go: complete test code

```go
func TestCreatePost(t *testing.T) {
	testCases := []struct {
		name          string
		mockClosure   func(m *mockCacheService)
		expectedError error
	}{
		{
			name:        "happy path",
			mockClosure: func(m *mockCacheService) {},
		},
		{
			name: "error when incrementing postCount key in cache",
			mockClosure: func(m *mockCacheService) {
				m.incrErr = errors.New("incr error")
			},
			expectedError: errors.New(`incrementing key "postCount": incr error`),
		},
	}
	for _, tc := range testCases {
		t.Run(tc.name, func(t *testing.T) {
			m := new(mockCacheService)
			tc.mockClosure(m)
			ps := NewPostService(m)
			err := ps.CreatePost(context.TODO(), &Post{})
			if err != nil {
				if tc.expectedError == nil {
					t.Fatalf("expected no error, got %v", err)
				}
				require.Equal(t, tc.expectedError.Error(), err.Error())
			} else {
				if tc.expectedError != nil {
					t.Fatalf("expected error to be %v, got nil", tc.expectedError)
				}
			}
		})
	}
}
```

Tiago Melo

github.com/tiagomelo

tiagoharris@gmail.com